

AI-Powered Crop Advisory & Disease Resolution Assistant

Simplified Hackathon POC Plan

1. POC Objective

Build a mobile-friendly AI crop assistant that allows a farmer to:

- Upload a crop image.
- Describe the farming problem through text or voice.
- Select crop, location and growth stage.
- Identify possible crop diseases, pest attacks, nutrient deficiencies or environmental stress.
- Receive treatment, irrigation and prevention guidance.
- Ask follow-up questions through a conversational bot.
- Receive the response in English, Hindi or Bengali.
- Listen to the recommendation through voice output.
- Review previous advisory cases.

The goal is to demonstrate all major AI capabilities visibly. Some features may use synthetic data, mock APIs, predefined rules or simulated outputs.

2. Simplified User Roles

Use a basic login screen with hardcoded credentials.

Role	Username	Password
Farmer	farmer	farmer123
Agriculture Officer	officer	officer123
Admin	admin	admin123

No JWT, identity provider or full authentication system is required.

The selected role can be stored in:

localStorage

The frontend will show different menus based on the logged-in role.

The backend may also receive a simple role value in the request header for demo-level API restrictions.

Example:

```
X-Demo-Role: farmer
```

3. Main Demo Journey

```
Farmer logs in
↓
Selects crop and growth stage
↓
Uploads affected crop image
↓
Describes symptoms through text or voice
↓
System analyzes visible image symptoms
↓
Rule engine compares symptoms with sample disease rules
↓
RAG retrieves agricultural recommendations
↓
Weather and soil data are fetched from mock APIs
↓
SLM generates an easy-to-understand advisory
↓
Guardrails validate the recommendation
↓
Response is translated and played through voice
↓
Case is saved in advisory history
```

4. Main Screens

Farmer Screens

1. Demo Login
2. Farmer Dashboard
3. New Crop Advisory

4. Crop Image Upload
5. Voice or Text Problem Description
6. Diagnosis and Recommendation
7. Conversational Follow-up
8. Advisory History

Agriculture Officer Screens

1. Escalated Cases
2. Case Review
3. Image and Diagnosis Details
4. Add Officer Recommendation
5. Mark Case as Resolved

Admin Screens

1. KPI Dashboard
2. Prompt Configuration
3. Knowledge Base
4. Synthetic Data Generator
5. Model Configuration
6. Agent and Tool Status
7. Logs and Exceptions

5. Crop Advisory Case Information

Each advisory case can contain:

- Farmer name.
- Crop name.
- Crop variety.
- Growth stage.
- Location.
- Symptom description.
- Uploaded crop image.
- Soil type.
- Recent weather.
- Previous fertilizer usage.
- Previous pesticide usage.
- Possible disease.
- Confidence score.
- Recommended actions.
- Preventive measures.
- Escalation status.

For the hackathon, many fields can be optional or selected from predefined dropdowns.

6. Implementation of the 20 Requested Areas

1. Data Synthesis

Create synthetic agricultural data for the demo.

Synthetic data examples

- Farmer profiles.
- Crop disease cases.
- Soil-test results.
- Weather records.
- Pest infestation cases.
- Fertilizer recommendations.
- Crop treatment history.
- Farmer feedback.

Demo feature

The Admin enters:

```
Generate 30 synthetic rice disease cases containing symptoms,  
weather, soil condition, disease name and recommendations.
```

The SLM returns structured JSON and stores it in SQLite.

Simplification

Provide predefined templates such as:

- Rice disease cases.
- Tomato disease cases.
- Potato pest cases.
- Soil nutrient cases.

2. Prompt Engineering

Provide a simple Prompt Configuration page.

Configurable prompts

- Crop diagnosis prompt.
- Recommendation prompt.
- Translation prompt.
- Guardrail prompt.
- Conversation prompt.

Example prompt structure

You are an agricultural advisory assistant.

Crop: {crop_name}

Growth stage: {growth_stage}

Farmer symptoms: {symptoms}

Image observations: {image_observations}

Weather: {weather_data}

Soil: {soil_data}

Retrieved guidance: {rag_context}

Generate simple and safe recommendations.

Store prompts in SQLite.

No complex prompt-versioning workflow is required.

3. RAG Types

Implement three simple RAG modes.

Basic RAG

Retrieve relevant agricultural guidance based on the farmer's question.

Metadata-Filtered RAG

Filter documents using:

- Crop.
- Disease.
- Language.
- Region.

Conversational RAG

Use previous chat messages to understand follow-up questions.

Example:

Farmer: My tomato leaves have brown spots.

Farmer: Can it spread to other plants?

The system should understand that the follow-up question refers to the tomato disease case.

Simulated RAG modes

Corrective RAG and Agentic RAG can be represented through an Admin toggle and execution trace without implementing a complex retrieval pipeline.

4. Agentic Tooling

Create a small set of tools.

Tools

- Analyze crop image.
- Search agricultural knowledge.
- Get weather information.
- Get soil information.
- Search previous cases.
- Translate response.
- Escalate case to officer.

UI execution trace

- ✓ Crop image analyzed
 - ✓ Disease rules checked
 - ✓ Agricultural knowledge searched
 - ✓ Weather retrieved
 - ✓ Recommendation generated
 - ✓ Safety check completed
-

5. Model Optimization

Use an SLM for most text-processing tasks.

SLM responsibilities

- Detect language.
- Identify crop and symptoms.
- Summarize conversation.
- Generate recommendations.
- Translate responses.
- Format the final answer.

Optimization techniques

- Keep prompts short.
- Retrieve only three or four document chunks.
- Summarize old conversation messages.
- Compress uploaded images.
- Cache weather results.
- Use structured JSON responses.
- Avoid sending unnecessary case history.

Dashboard indicators

- Response time.
 - Input tokens.
 - Output tokens.
 - Retrieved chunks.
 - SLM calls.
 - Estimated cost.
-

6. Multi-Agent System

Use a lightweight multi-agent flow.

Supervisor Agent

Decides which agents or tools are required.

Image Analysis Agent

Extracts visible crop symptoms from the image.

Disease Rule Agent

Matches visible symptoms with predefined disease rules.

RAG Agent

Retrieves relevant agricultural recommendations.

Weather and Soil Agent

Fetches weather and soil context from mock services.

Advisory Agent

Combines all information and generates recommendations using the SLM.

Guardrail Agent

Checks safety, confidence and unsupported recommendations.

Translation and Voice Agent

Translates the final answer and prepares voice output.

Simplified agent flow

```
flowchart TD
    A[Farmer Input] --> B[Supervisor Agent]

    B --> C[Image Analysis Agent]
    B --> D[Disease Rule Agent]
    B --> E[RAG Agent]
    B --> F[Weather and Soil Agent]

    C --> G[Advisory Agent]
    D --> G
    E --> G
    F --> G

    G --> H[Guardrail Agent]
    H --> I[Translation and Voice Agent]
    I --> J[Farmer Response]
```

The agents can be normal Python service classes. LangGraph is optional.

7. MCP Protocol

Implement one lightweight local MCP server.

MCP tools

- `get_weather`
- `get_soil_profile`
- `search_crop_guidance`
- `create_officer_escalation`

The tools can return synthetic data from SQLite.

Architecture

```
FastAPI Application
  |
  MCP Client
  |
Local MCP Server
  |
SQLite and Mock Services
```

The Admin page can show:

- MCP server status.
- Available tools.
- Last execution.
- Execution time.
- Success or failure.

8. RBAC

Use hardcoded credentials instead of a complete authentication system.

Farmer access

- Submit crop problem.
- Upload crop image.
- Ask questions.
- View own advisory history.

Officer access

- View escalated cases.

- Add recommendations.
- Mark cases as resolved.

Admin access

- Configure prompts.
- Upload knowledge documents.
- Generate synthetic data.
- View logs.
- View KPI dashboard.

Simplification

Role information can be handled through:

```
React localStorage
```

and a simple request header.

No JWT, password encryption, refresh token or user-management module is required.

9. Conversational Bot

Features

- Chat-style interface.
- Ask questions through text.
- Ask questions through voice.
- Upload image inside chat.
- Conversation history.
- Follow-up question support.
- Suggested follow-up questions.
- Copy response.
- Read response aloud.
- Farmer feedback.

Suggested questions

- What should I do immediately?
 - Can the disease spread?
 - Is there an organic treatment?
 - Should I reduce irrigation?
 - When should I upload another image?
 - Should I contact an agricultural officer?
-

10. API Integration

Use one real API and several mock APIs.

Recommended integration

Use a weather API for current or recent weather information.

Mock APIs

- Soil information.
- Crop information.
- Local treatment availability.
- Previous disease cases.
- Agriculture officer escalation.

Example endpoints

```
GET /api/weather
GET /api/soil/{location}
GET /api/crops/{crop}/diseases
GET /api/treatments
POST /api/escalations
```

Mock APIs can return data from SQLite or JSON files.

11. Logs and Exceptions

Store basic execution logs in SQLite.

Log fields

- Case ID.
- User role.
- Agent name.
- Tool name.
- Status.
- Response time.
- SLM used.
- Error message.
- Timestamp.

Common error cases

- Invalid image.

- Image analysis failure.
- Weather API unavailable.
- No RAG result.
- SLM timeout.
- Translation failure.
- Voice processing failure.
- Low-confidence diagnosis.

User-friendly error

```
{  
  "code": "IMAGE_NOT_CLEAR",  
  "message":  
    "The crop image is not clear. Please upload another image in better light."  
}
```

12. Agent KPI Dashboard

KPI cards

- Total advisory cases.
- Total crop images analyzed.
- Resolved cases.
- Escalated cases.
- Average diagnosis confidence.
- Average response time.
- Successful tool calls.
- Failed tool calls.
- Farmer feedback score.
- Languages used.

Charts

- Cases by crop.
- Common diseases.
- Disease reports by location.
- Agent execution time.
- SLM usage.
- Cases by language.
- Escalation trend.
- Farmer satisfaction trend.

Use Recharts in React.

13. UI Frameworks

Recommended frontend stack

```
React
JavaScript or TypeScript
Vite
Material UI
React Router
Axios
React Hook Form
Recharts
React Markdown
```

UI design recommendations

- Mobile-first layout.
 - Large buttons.
 - Camera-friendly upload.
 - Simple forms.
 - Minimal typing.
 - Voice button.
 - Language selector.
 - Clear diagnosis severity.
 - Easy-to-read recommendation cards.
-

14. API Stack

Recommended API stack

```
FastAPI
Pydantic
SQLAlchemy
Uvicorn
HTTPX
Python multipart
```

Optional packages

```
LangGraph
FastMCP
```

FAISS
Chroma

API approach

- Use `/api/v1` routes.
- Use simple JSON requests and responses.
- Use Pydantic validation.
- Use standard error responses.
- Streaming responses are optional.

15. Backend Stack

Recommended backend

Python
FastAPI
SQLite
SQLAlchemy
FAISS or Chroma
Local file storage
SLM provider
Vision-capable model
FastMCP

Simplified backend modules

demo_login
crop_advisory
chat
image_analysis
disease_rules
rag
weather
soil
agents
tools
mcp
prompts
guardrails
translation
voice

```
analytics
logs
```

Use a single FastAPI application instead of microservices.

16. Guardrails and PII

PII fields

- Farmer name.
- Phone number.
- Email.
- Exact farm location.
- Voice recording.
- Image metadata.

POC guardrails

- Mask phone numbers.
- Mask email addresses.
- Remove image metadata where possible.
- Do not log full personal information.
- Reject unrelated or unsafe prompts.
- Display diagnosis as a possible condition.
- Display confidence score.
- Avoid generating unsupported pesticide dosages.
- Recommend checking local agricultural guidelines.
- Escalate low-confidence cases.

Example

```
Original:
My phone number is 9876543210.
```

```
Stored:
My phone number is [PHONE].
```

17. Multilingual

Support three demo languages:

- English.
- Hindi.

- Bengali.

Features

- Manual language selection.
- Basic automatic language detection.
- Translate farmer questions.
- Translate recommendations.
- Voice playback in the selected language.

Processing flow

```
Farmer input
↓
Language detection
↓
Translation when required
↓
SLM and RAG processing
↓
Translate final recommendation
↓
Text and voice response
```

18. Multimodal

Inputs

- Crop image.
- Text description.
- Voice description.
- Soil-report PDF.
- Soil-report image.

Main demo

The farmer uploads:

- A leaf image.
- A voice description.
- Crop and location details.

The system combines all three inputs before generating the recommendation.

19. SLM

Use an SLM as the primary language model.

SLM tasks

- Intent detection.
- Language detection.
- Symptom extraction.
- Conversation summarization.
- RAG answer generation.
- Recommendation generation.
- Translation.
- Response formatting.

Admin configuration

- SLM name.
- Temperature.
- Maximum tokens.
- Active status.
- Fallback model.
- Context size.

Simplification

A cloud-hosted small model may be used as an SLM substitute if a local model is difficult to configure during the hackathon.

20. Voice Interface

Features

- Microphone button.
- Record symptom description.
- Convert speech to text.
- Allow transcript editing.
- Send transcript to the advisory bot.
- Play the recommendation as audio.
- Pause and replay response.

Simplest implementation

Use browser features:

Web Speech API
speechSynthesis API

This avoids a separate speech-service integration.

7. Hybrid Crop Image Analysis

Use Option C: Hybrid Demonstration.

Step 1: Vision Analysis

A vision-capable model examines the crop image and extracts visible symptoms.

Example output:

```
{  
  "leaf_color": "yellow-green",  
  "spots": "brown circular spots",  
  "spot_location": "older leaves",  
  "wilting": false,  
  "visible_pest": false  
}
```

Step 2: Disease Rule Matching

A simple Python rule engine compares the extracted symptoms with predefined disease rules.

Example:

```
Crop: Tomato  
Brown circular spots: Yes  
Older leaves affected: Yes  
Yellow area around spots: Yes  
  
Possible match: Early Blight  
Rule confidence: 80%
```

Step 3: RAG Retrieval

Retrieve recommendations from agricultural documents using:

- Crop name.
- Matched disease.
- Extracted symptoms.
- Region.
- Growth stage.

Step 4: Weather and Soil Context

Add synthetic or API-based information:

```
Recent humidity: High  
Recent rainfall: Heavy  
Soil drainage: Moderate
```

Step 5: SLM Advisory

The SLM combines:

- Image observations.
- Rule-based disease match.
- RAG guidance.
- Weather.
- Soil.
- Farmer description.

It then generates a simple advisory.

Benefits of the hybrid approach

- Vision model demonstrates multimodal AI.
 - Rule engine makes the diagnosis explainable.
 - RAG provides grounded recommendations.
 - SLM creates a farmer-friendly answer.
 - No custom computer-vision model training is required.
-

8. Simplified Architecture

flowchart TB

U[Farmer / Officer / Admin] --> UI[React Mobile-Friendly UI]

UI --> API[FastAPI Application]

API --> ROLE[Hardcoded Login and Role Check]

API --> CHAT[Conversational Bot]

API --> CASE[Crop Advisory Service]

API --> ADMIN[Admin Configuration]

API --> KPI[KPI and Logs]

CHAT --> ORCH[Simple Agent Orchestrator]

ORCH --> VISION[Vision Symptom Extraction]

ORCH --> RULES[Disease Rule Engine]

ORCH --> RAG[RAG Search]

ORCH --> WEATHER[Weather Tool]

ORCH --> SOIL[Mock Soil Tool]

RAG --> VECTOR[FAISS or Chroma]

RAG --> DOCS[Agricultural Documents]

WEATHER --> MCP[MCP Server]

SOIL --> MCP

VISION --> SLM[SLM Advisory Generator]

RULES --> SLM

RAG --> SLM

WEATHER --> SLM

SOIL --> SLM

SLM --> GUARD[Guardrail Check]

GUARD --> LANG[Translation and Voice]

LANG --> CHAT

CASE --> DB[(SQLite)]

ADMIN --> DB

KPI --> DB

9. Suggested Project Structure

```
crop-advisory-poc/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   │   ├── Login/
│   │   │   ├── FarmerDashboard/
│   │   │   ├── NewAdvisory/
│   │   │   ├── Chat/
│   │   │   ├── AdvisoryHistory/
│   │   │   ├── OfficerQueue/
│   │   │   ├── AdminDashboard/
│   │   │   ├── PromptSettings/
│   │   │   ├── KnowledgeBase/
│   │   │   └── Logs/
│   │   ├── services/
│   │   └── routes/
│   └── package.json
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── agents/
│   │   ├── tools/
│   │   ├── rules/
│   │   ├── rag/
│   │   ├── mcp/
│   │   └── database/
│   ├── uploads/
│   ├── knowledge/
│   ├── vector_store/
│   └── synthetic_data/
├── requirements.txt
├── .env.example
└── README.md
```

10. Simplified SQLite Tables

```
demo_users
farmer_profiles
crop_catalog
disease_rules

advisory_cases
case_images
case_diagnoses
case_recommendations
case_feedback
case_escalations

conversations
messages
conversation_summaries

documents
document_chunks

prompt_settings
model_settings

agent_logs
tool_logs
application_logs
guardrail_events
```

11. Suggested API Endpoints

Demo login

```
POST /api/v1/demo-login
```

Advisory

```
POST /api/v1/advisories
GET /api/v1/advisories
GET /api/v1/advisories/{case_id}
```

```
POST /api/v1/advisories/{case_id}/image
POST /api/v1/advisories/{case_id}/analyze
POST /api/v1/advisories/{case_id}/feedback
POST /api/v1/advisories/{case_id}/escalate
```

Chat

```
POST /api/v1/chat
GET /api/v1/conversations
GET /api/v1/conversations/{conversation_id}
```

Knowledge base

```
POST /api/v1/knowledge/upload
GET /api/v1/knowledge/documents
DELETE /api/v1/knowledge/documents/{document_id}
POST /api/v1/knowledge/reindex
```

Admin

```
GET /api/v1/admin/prompts
PUT /api/v1/admin/prompts

GET /api/v1/admin/model
PUT /api/v1/admin/model

GET /api/v1/admin/logs
GET /api/v1/admin/kpis
```

12. Hackathon Implementation Priority

Priority 1: Required Working Flow

- Hardcoded login.
- Farmer and Admin roles.
- Crop selection.
- Image upload.
- Text symptom input.
- Hybrid image analysis.

- Basic RAG.
- SLM recommendation.
- Advisory result page.
- Save case in SQLite.

Priority 2: Demo Differentiators

- Voice input.
- Voice output.
- Bengali and Hindi responses.
- Weather API.
- Agent execution trace.
- Guardrail validation.
- Officer escalation.

Priority 3: Dashboard Features

- KPI dashboard.
- Logs.
- Prompt settings.
- Knowledge upload.
- Synthetic data generator.
- MCP status screen.

Simulated Features

- Advanced disease prediction.
- Real soil integration.
- Local fertilizer inventory.
- Complex officer workflow.
- Production-level disease model.
- Offline mobile synchronization.

13. Suggested Hackathon Demo

Step 1

Log in as:

farmer / farmer123

Step 2

Select:

Crop: Tomato
Growth stage: Fruiting
Language: Bengali

Step 3

Upload a sample diseased tomato-leaf image.

Step 4

Record or type the symptom description.

Step 5

Display the execution trace:

Image symptoms extracted
Disease rules checked
Weather retrieved
Agricultural guidance searched
SLM advisory generated
Guardrail validation completed
Response translated into Bengali

Step 6

Show:

- Possible disease.
- Confidence.
- Visible symptoms.
- Immediate action.
- Treatment options.
- Irrigation guidance.
- Preventive measures.
- Retrieved sources.
- Listen-to-response button.

Step 7

Ask a follow-up question:

Can this disease spread to nearby plants?

Step 8

Log in as:

admin / admin123

Show:

- KPI dashboard.
- Prompt settings.
- Knowledge documents.
- Agent logs.
- MCP tool status.
- Synthetic data generator.

14. Final Recommended Hackathon Stack

```
React
Material UI
FastAPI
SQLite
FAISS or Chroma
Vision-capable model
SLM
Simple Python disease-rule engine
Browser voice APIs
FastMCP
Mock weather and soil services
```

The POC should focus on one complete and visually impressive farmer journey rather than technically completing every enterprise feature.

The main differentiator should be the hybrid crop-diagnosis workflow:

Crop Image

- + Vision Symptom Extraction
- + Disease Rule Matching
- + Agricultural RAG
- + Weather and Soil Context
- + SLM Recommendation
- + Multilingual Voice Response